

Инструкция по использованию и портированию BLCL Bootloader библиотеки

Введение

Эта библиотека реализует bootloader для STM32 микроконтроллеров с поддержкой обновления прошивки через RS485/UART. Она использует протокол с пакетами:

- Стартовый байт: 0xAA
- Длина пакета
- Команда
- Данные
- CRC32

Bootloader проверяет наличие команды обновления (TryConnection 0x7E) и, если её нет, переходит в основное приложение (main app).

Структура библиотеки

blcl_bootloader.h

Заголовочный файл с конфигурацией и функциями. Здесь определяется структура BLCL_Config_t для настройки bootloader и объявляются функции:

- BLCL_Init() — инициализация bootloader
- BLCL_Run() — основной цикл (ожидание обновления или переход в приложение)

blcl_bootloader.c

Исходный код реализации (интегрируйте в ваш проект). Содержит внутренние функции:

Функция	Назначение
create_blcl_packet()	Создание пакетов для отправки
receive_packet()	Приём и разбор пакетов
calc_crc()	Расчёт CRC32 для проверки целостности
erase() / program()	Работа с Flash-памятью
delay_us()	Задержка на базе DWT

Функция	Назначение
<code>send_ack()</code> / <code>send_nack()</code>	Отправка подтверждений
<code>jump_to_app()</code>	Переход в основное приложение
<code>send_version_date()</code>	Отправка информации о версии
<code>write_version_date()</code>	Запись метаданных в Flash
Обработчики команд	<code>WriteAddress 0x01</code> , <code>WriteData 0x03</code> , <code>SetFirmwareInfo 0x05</code> , <code>TryConnection 0x7E</code>

main.c проекта

Инициализация периферии и запуск bootloader:

1. Инициализация периферии:

- GPIO
- UART
- Clock

2. Заполнение структуры конфигурации

3. Вызов bootloader:

```
BLCL_Init(&config);  
BLCL_Run(&config);
```

Использование библиотеки

1. Вставьте библиотеку в проект STM32CubeIDE (или аналогичный)
2. Настройте конфигурацию в `main.c`
3. Вызовите `BLCL_Init()` и `BLCL_Run()`

Примечание: Библиотека использует HAL, поэтому сгенерируйте базовый проект через STM32CubeMX для вашего МК.

Структура конфигурации `BLCL_Config_t`

Bootloader настраивается через структуру `BLCL_Config_t` , которая передаётся в `BLCL_Init()` и `BLCL_Run()` . Пример заполнения:

```
// Конфигурация bootloader
BLCL_Config_t config = {
    .huart = &huart1,           // Handle UART (e.g., &huart1)
    .de_gpio = USART_DE_GPIO_Port, // Порт DE (e.g., GPIOB)
    .de_pin = USART_DE_Pin,       // Пин DE (e.g., GPIO_PIN_5)
    .app_address = 0x08008000,    // Адрес начала main app во Flash
    .flash_size = 0x20000,       // Полный размер Flash в байтах (e.g., 128KB)
    .init_timeout_ms = 2000,     // Таймаут на initial TryConnection (ms)
    .packet_timeout_ms = 10000,  // Таймаут на пакеты в цикле обновления (ms)
    .try_cmd = 0x7E,             // Команда TryConnection (e.g., 0x7E)
    .rs485_delay_us = 100,       // Задержка для RS485 в микросекундах
    .metadata_address = 0x0801F000, // Адрес метаданных (version/date) во Flash
    .metadata_page_size = 0x800  // Размер страницы Flash для метаданных (e.g., 2KB)
};

// Инициализация bootloader
if (BLCL_Init(&config) != HAL_OK) {
    Error_Handler();
}

// Запуск основного цикла
BLCL_Run(&config);
```

Пояснение полей конфигурации

Поле	Назначение	Зачем нужно
huart	Handle UART (например, &huart1)	Указывает на инициализированный UART для передачи/приёма пакетов
de_gpio	Порт DE (например, GPIOB)	Управление RS485 (Driver Enable)
de_pin	Пин DE (например, GPIO_PIN_5)	Конкретный пин для переключения режима передачи/приёма RS485
app_address	Адрес начала main app во Flash (например, 0x08008000)	Bootloader переходит по этому адресу, если нет обновления
flash_size	Полный размер Flash в байтах (например, 0x20000 для 128KB)	Проверка переполнения при стирании/записи
init_timeout_ms	Таймаут на initial TryConnection (2000 ms)	Время ожидания ответа хоста перед переходом в app
packet_timeout_ms	Таймаут на пакеты в цикле обновления (10000 ms)	Прерывание update при таймауте и переход в app

Поле	Назначение	Зачем нужно
try_cmd	Команда TryConnection (0x7E)	Код команды для инициации обновления
rs485_delay_us	Задержка для RS485 (100 μs)	Пауза после переключения DE для избежания коллизий
metadata_address	Адрес метаданных (например, 0x0801F000)	Хранение версии/даты firmware (64 байта)
metadata_page_size	Размер страницы Flash для метаданных (например, 0x800 для 2KB)	Для стирания метаданных без повреждения основной памяти

Работа с метаданными (версия и дата)

Bootloader хранит метаданные (версию прошивки и дату) во Flash по адресу `.metadata_address`. Структура метаданных занимает 64 байта:

- Первые 32 байта: версия в формате строки (ASCII)
- Следующие 32 байта: дата в формате строки (ASCII)
- Разделитель: нулевой байт (`\0`) в конце каждой строки

Инициализация метаданных

Если метаданные пусты (все байты равны `0xFF`), bootloader автоматически устанавливает значения по умолчанию:

- Версия: "v1.0.0"
- Дата: "2025-12-20"

Вывод версии и даты

Bootloader автоматически читает и отправляет метаданные по UART после запуска:

```
Version: v1.0.0 Date: 2025-12-20\r\n
```

Функция `send_version_date()` :

```
void send_version_date(BLCL_Config_t* config, const char* version, const char* date)
```

Отправляет форматированную строку через `send_string()`.

Изменение версии и даты

Команда: `0x05` (`SetFirmwareInfo`) — позволяет хосту обновить метаданные.

Формат данных пакета:

```
version\0date\0
```

где:

- `version` — строка версии (макс. 31 байт + нулевой терминатор)
- `date` — строка даты (макс. 31 байт + нулевой терминатор)
- `\0` — нулевой байт-разделитель
- Общая длина данных ≤ 64 байта

Логика обработки:

1. Если строка версии пустая → используется значение по умолчанию `"v1.0.0"`
2. Если строка даты пустая → используется значение по умолчанию `"2025-12-20"`
3. Bootloader стирает страницу `metadata`
4. Записывает новые данные
5. Отправляет `АСК` (если успешно) или `НАСК` (при ошибке)

Тайминг команды:

- Работает в начальном таймауте (5 секунд после запуска)
- Или в режиме обновления (после получения `TryConnection`)

Пример пакета обновления метаданных

Для обновления версии на `"v2.1.0"` и даты на `"2026-01-01"` :

Структура пакета:

```
0xAA // Стартовый байт
0x12 0x00 // Длина данных (18 байт)
0x05 // Команда SetFirmwareInfo
// Данные (18 байт):
'v' '2' '.' '1' '.' '0' 0x00 // "v2.1.0\0" (7 байт)
'2' '0' '2' '6' '-' '0' '1' '-' '0' '1' 0x00 // "2026-01-01\0" (11 байт)
0xFF 0xFF 0xFF 0xFF // CRC32
```

Пример:

AA 12 00 05 76 32 2E 31 2E 30 00 32 30 32 36 2D 30 31 2D 30 31 00 F8 E6 BC 64

Разбор пакета:

Позиция	Значение (hex)	Описание
0	0xAA	Стартовый байт
1-2	0x12 0x00	Длина данных: младший байт 0x12 (18), старший байт 0x00
3	0x05	Команда: SetFirmwareInfo
4-21	0x76...0x00	Данные (18 байт)
22-25	0xF8 0xE6 0xBC 0x64	CRC32: 0x64BCE6F8

Данные в ASCII:

Байты 4-10: 0x76 0x32 0x2E 0x31 0x2E 0x30 0x00 → "v2.1.0\0"

Байты 11-21: 0x32 0x30 0x32 0x36 0x2D 0x30 0x31 0x2D 0x30 0x31 0x00 → "2026-01-01\0"

Ответ bootloader:

При успешном обновлении отправляется пакет аск :

0xAA 0x02 0x00 0x05 0x00 0x00 CRC32

где:

- 0x05 — команда (эхо)
- 0x00 — код ошибки (успех)

Методика расчета CRC

Алгоритм CRC32

- Полином: 0xEDB88320 (reflected, как в Ethernet/ZIP)
- Инициализация: crc = 0xFFFFFFFF
- Финальный XOR: crc ^= 0xFFFFFFFF

Формула расчета для каждого байта:

```
crc = CRC_TABLE[(crc ^ byte) & 0xFF] ^ (crc >> 8)
```

Данные для расчета CRC:

- Включаются: длина (2 байта) + команда (1 байт) + данные (N байт)
- Исключаются: стартовый байт 0xAA и само поле CRC

Генерация таблицы CRC_TABLE:

```
void generate_crc_table(void) {
    for (uint32_t i = 0; i < 256; i++) {
        uint32_t c = i;
        for (uint8_t j = 0; j < 8; j++) {
            c = (c & 1) ? (c >> 1) ^ 0xEDB88320 : (c >> 1);
        }
        CRC_TABLE[i] = c;
    }
}
```

Портирование на другой STM32 контроллер

Bootloader использует HAL, поэтому портирование сводится к адаптации под конкретный МК:

Ключевые аспекты портирования:

- Размер Flash-памяти
- Размер страницы/сектора Flash
- Экземпляр UART
- Тактовая частота
- Поддержка DWT (для задержек)
- Тип программирования Flash (32-битное или 64-битное)

1. Измените HAL заголовок в `blcl_bootloader.h` :

- Замените `#include "stm3214xx_hal.h"` на соответствующий для вашего MCU, например, `#include "stm32f4xx_hal.h"` или `#include "stm32g4xx_hal.h"` . Это из CubeMX генератора проекта.
- Если нужно, обновите core header: для Cortex-M4 — `#include "core_cm4.h"` , для M3 — `#include "core_cm3.h"` , для M0 — `#include "core_cm0.h"` .

2. Адаптируйте размер страницы flash в `erase`:

- В `BLCL_Run` , в обработке `cmd 0x01 (WriteAddress)`, в структуре `FLASH_EraseInitTypeDef` :

- `erase.Page = (addr - 0x08000000) / 0x800; — 0x800 = 2KB для STM32L4/G4.`

Проверьте datasheet вашего MCU и замените:

- STM32F1: 1KB (0x400).
- STM32F4: 16KB-128KB сектора (используйте логику для расчета sector:
`erase.Sector = GetSector(addr);` — добавьте функцию из HAL примеров).
- STM32H7: 128KB сектора (0x20000).
- `erase.NbPages = (size + 0x7FF) / 0x800;` — Адаптируйте под размер страницы/сектора (замените 0x7FF и 0x800 соответственно).
- `erase.Banks = FLASH_BANK_1;` — Для dual-bank MCU (e.g., STM32H7/L4 с >512KB) оставьте или добавьте `FLASH_BANK_1 | FLASH_BANK_2` если адрес в BANK2; для single-bank удалите.
- `erase.TypeErase = FLASH_TYPEERASE_PAGES;` — Для серий с sectors (F1/F2/F4/G4) замените на `FLASH_TYPEERASE_SECTORS;` .
- Аналогично адаптируйте `erase` в `write_version_date` .
- Также обновите `.flash_size` в конфиге (в байтах, e.g., 0x80000 для 512KB) и `.metadata_page_size` (e.g., 0x4000 для 16KB сектора в F4).

3. Настройте UART и GPIO:

- В `main.c` : В `MX_USART1_UART_Init` измените `huart1.Instance = USART1;` на ваш UART (e.g., `USART2`).
- `BaudRate`, `Mode` и т.д. оставьте, если 115200 8N1 подходит; иначе адаптируйте.
- В `MX_GPIO_Init` : `USART_DE_GPIO_Port` и `USART_DE_Pin` — адаптируйте под ваш DE пин для RS485 (в CubeMX настройте как Output PP, Low speed).
- В конфиге `BLCL_Config_t` : `.huart = &huart1;` — измените на ваш handle (e.g., `&huart2`); `.de_gpio` и `.de_pin` под ваш пин.

4. Clock configuration:

- В `SystemClock_Config` : Адаптируйте под ваш MCU (HSE, PLL, dividers). Используйте CubeMX для генерации.
- Убедитесь, что `SystemCoreClock` установлен правильно для `delay_us` (использует DWT с `SystemCoreClock / 1000000`).

5. IRQ и cleanup в `jump_to_app` :

- В `jump_to_app` : `NVIC_ClearPendingIRQ(USART1_IRQn); NVIC_DisableIRQ(USART1_IRQn);` — Замените `USART1_IRQn` на ваш UART IRQ (e.g., `USART2_IRQn`).
- Если `SysTick` не используется, оставьте; иначе добавьте аналогичную очистку для других IRQ.

6. App address и flash check:

- `.app_address = 0x08008000`; — Адаптируйте под размер bootloader (обычно bootloader в начале flash ~32KB, app после). В linker script резервируйте место (см. ниже в разделе main app).
- В `jump_to_app`: Проверка `(*(__IO uint32_t*)app_address) != 0xFFFFFFFF` — работает для erased flash; если ваш MCU по-другому (e.g., 0x00 в erased), адаптируйте на валидный stack pointer.

7. Delay для RS485:

- `.rs485_delay_us = 100`; — Адаптируйте под вашу аппаратную часть (turnaround time). Тестируйте с осциллографом.
- Для Cortex-M0 (без DWT): В `BLCL_Init` удалите DWT init; в `delay_us` замените на `HAL_Delay` или NOP цикл (e.g., `for(volatile uint32_t i=0; i<us*(SystemCoreClock/1000000)/3; i++)`; — калибруйте).

8. Другие:

- В `BLCL_Init`: DWT init работает для Cortex-M3/M4/M7+; для M0 используйте другой delay (см. выше).
- Если MCU без floating point, избегайте float (тут нет).
- Для program в `BLCL_Run` (cmd 0x03): `FLASH_TYPEPROGRAM_DOUBLEWORD` (64-bit) для M4/M7; для M3 (F1/F3) замените на `FLASH_TYPEPROGRAM_WORD` (32-bit) и пишите по 4 байта.
- Тестируйте: Скомпилируйте, прошейте bootloader через ST-Link, проверьте jump в app без обновления, затем обновление через RS485 (используйте инструмент для отправки пакетов).

Изменения в основном приложении (main app)

Main app — это ваша основная прошивка, которая стартует после bootloader. Bootloader прыгает в app по адресу `.app_address` (e.g., 0x08008000). Чтобы app работала корректно (векторы прерываний, стек, периферия), внесите изменения в проект app. App компилируется отдельно от bootloader.

1. Linker script (ld файл):

- В STM32CubeIDE найдите файл linker script (обычно `STM32xxxx_FLASH.ld`, где xxxx — ваш MCU, e.g., `STM32L476RG_FLASH.ld`).
- В секции `MEMORY` замените или добавьте: `FLASH (rx) : ORIGIN = 0x08008000, LENGTH = 0x20000 - 0x8000` /* Пример для 128KB флеша минус 32KB под bootloader; адаптируйте под ваш `.flash_size` */

- **Что заменить:** Найдите строку `FLASH (rx) : ORIGIN = 0x08000000, LENGTH = ...` и измените ORIGIN на ваш `.app_address` (e.g., 0x08008000), LENGTH — остаток флеша.
- Для векторной таблицы: В файле `system_stm32xxxx.c` (e.g., `system_stm32l4xx.c`) найдите `#define VECT_TAB_OFFSET 0x00` и замените на `#define VECT_TAB_OFFSET 0x8000UL` (смещение в

байтах, e.g., 0x8000 для 32KB bootloader). Это автоматически сместит VTOR в `SystemInit()`.

2. Vector table relocation:

- Альтернатива linker: В `main.c` app перед вызовом `HAL_Init()` вставьте: `SCB->VTOR = 0x08008000; // Ваш .app_address`, чтобы векторная таблица указывала на начало app
- Где вставить: После `#include` и перед `HAL_Init()`; в функции `main()`.
- Или в `system_stm32xxxx.c`: В функции `SystemInit()` найдите `SCB->VTOR = FLASH_BASE | VECT_TAB_OFFSET`; и убедитесь, что `VECT_TAB_OFFSET` установлен (как в п.1). Если нет — добавьте строку `SCB->VTOR = 0x08008000UL`; вручную после стандартного кода.
- Зачем: Bootloader устанавливает VTOR перед прыжком, но app должно подтвердить, чтобы прерывания работали правильно (иначе краш при IRQ).

3. Stack pointer:

- В `jump_to_app` bootloader устанавливает `__set_MSP((__IO uint32_t*)app_address)`; — Убедитесь, что в app linker script секция `.isr_vector` в начале FLASH (стандартно). Ничего не меняйте в коде app.

4. Периферия:

- Инициализируйте UART, GPIO, Clock заново в app, так как bootloader деинициализирует их перед jump.
- В `main.c` app: Вызовите `MX_USART1_UART_Init()`; , `MX_GPIO_Init()`; и `SystemClock_Config()`; как обычно (CubeMX генерирует).

5. Build и bin файл:

- Скомпилируйте app с адаптированным linker.
- Для обновления: Конвертируйте `.elf` в `.bin` (в CubeIDE post-build: `arm-none-eabi-objcopy -O binary ${ProjName}.elf ${ProjName}.bin`).
- В `bin` добавьте CRC32 в конец (bootloader ожидает data + 4-byte CRC для верификации; рассчитайте через код или инструмент).

6. Тестирование:

- Прошейте bootloader в 0x08000000 через ST-Link.
- Прошейте app в 0x08008000 (используйте ST-Link с offset или bootloader для обновления).
- Без RS485 команды — должен перейти в main app.
- С обновлением — стереть/записать main app flash, проверить CRC.